

LiteasyClaw Agent Architecture Audit

1. 审计结论

LiteasyClaw 当前不是简单聊天应用，而是一个桌面学术工作台内嵌 Agent 系统。现有架构已经形成四层：

1. `agent-api`：统一会话、run、事件、确认、取消、CLI/MCP 入口。
2. `agent-core`：每轮运行前装配 `agent.md`、`memory`、能力摘要、预算与运行时上下文。
3. `agent-runtime`：把命令模式自然语言规划为 `SemanticActionPlan`，做契约校验、策略决策、确认和 action 执行。
4. 领域能力层：`actions` / `skills` / `retrieval` / `models` / `artifacts` / `generative-ui` / `workspace` 等。

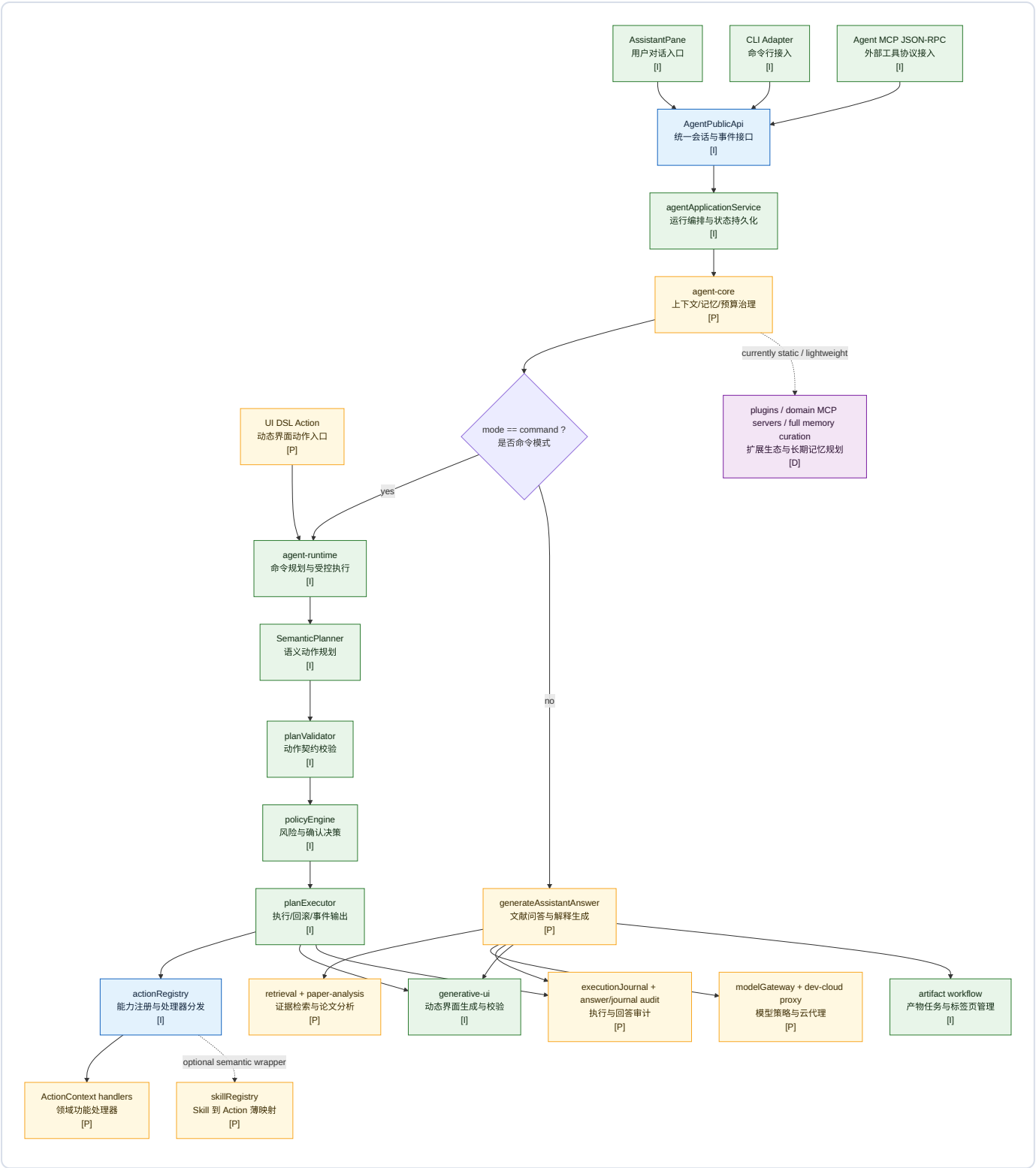
关键判断：

- `Command Mode` 已具备可审计执行链路：自然语言不能直接改状态，必须先变成注册 action，再通过 `schema`、`policy`、`confirmation`。
- `QA / Explain` 已具备证据检索、模型生成、审计和 UI DSL 输出链路，但检索仍偏本地 chunk 与 `demo fallback`，不是完整生产级 RAG。
- `agent-core` 的配置、`memory`、`budget` 已进入真实运行路径，但 `agent.md`、`skills`、`plugins`、`MCP`、`memory` 治理多数仍是静态配置或轻量实现。
- `skills` 现在更像“能力语义封装和文档目录”，真正执行边界仍是 `actionRegistry`。
- `agent-api` 已经成为前端、CLI、MCP 的统一外壳，但 `AssistantPane` 里仍保留部分本地 `runtime / dynamic UI action` 兼容路径，需要继续收敛。

2. 成熟度标记

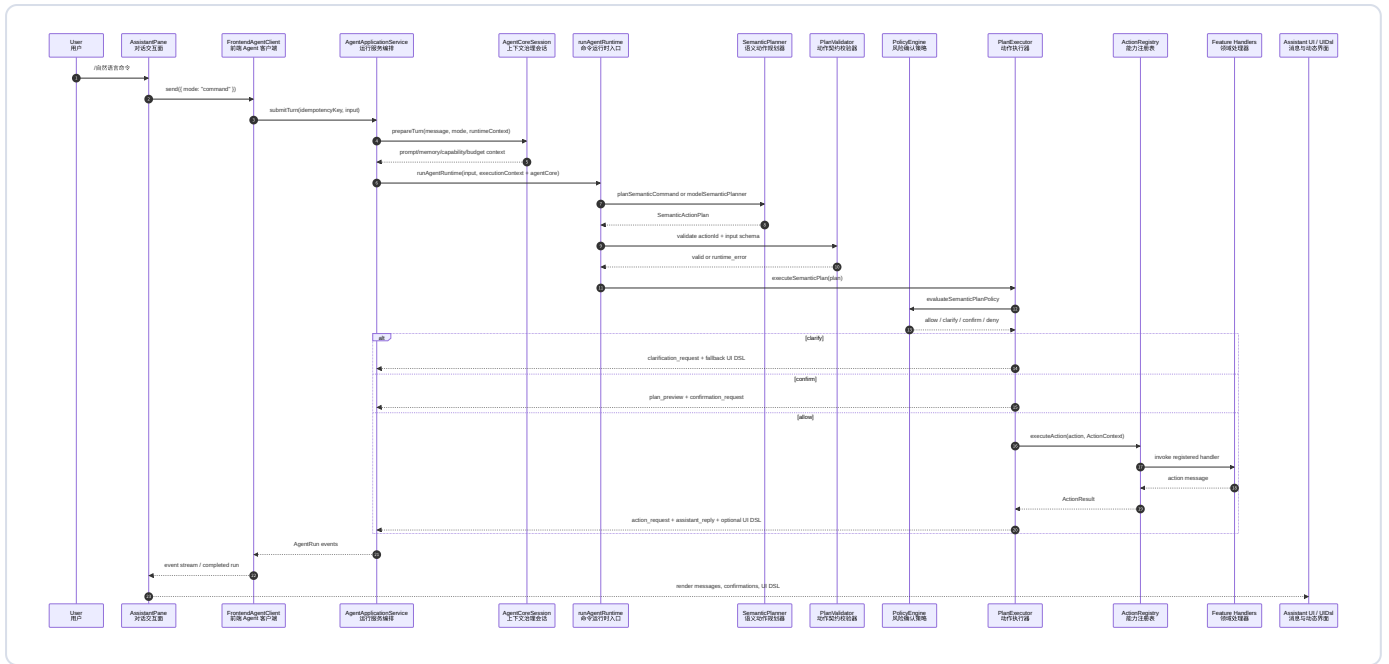
- [I] Implemented：当前源码已接入主链路，可被 UI/API 调用
- [P] Partial：主结构存在，但能力、治理、持久化或处理面不完整
- [D] Design/Planned：仅文档、默认配置或未来接口，尚未形成真实执行能力

3. Agent 总览图



4. Command Mode 执行链路

Command Mode 是当前最像 “Agent 操作系统” 的链路。它的安全假设是：自然语言、模型输出和 UI DSL 都不可信，必须降级为受控 action。

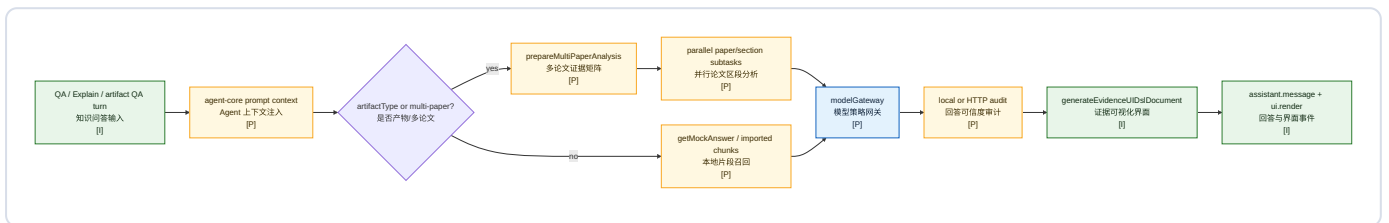


实现锚点：

- LiteasyClaw/desktop/src/app/features/agent-runtime/runtimeOrchestrator.ts
- LiteasyClaw/desktop/src/app/features/agent-runtime/modelSemanticPlanner.ts
- LiteasyClaw/desktop/src/app/features/agent-runtime/planValidator.ts
- LiteasyClaw/desktop/src/app/features/agent-runtime/policyEngine.ts
- LiteasyClaw/desktop/src/app/features/agent-runtime/planExecutor.ts
- LiteasyClaw/desktop/src/app/features/skills/actionRegistry.ts

5. QA / Explain 知识链路

QA / Explain 不是 state-changing command，但仍经过 `agent-core.prepareTurn()`。它主要服务文献理解、比较和 artifact 生成前的分析。

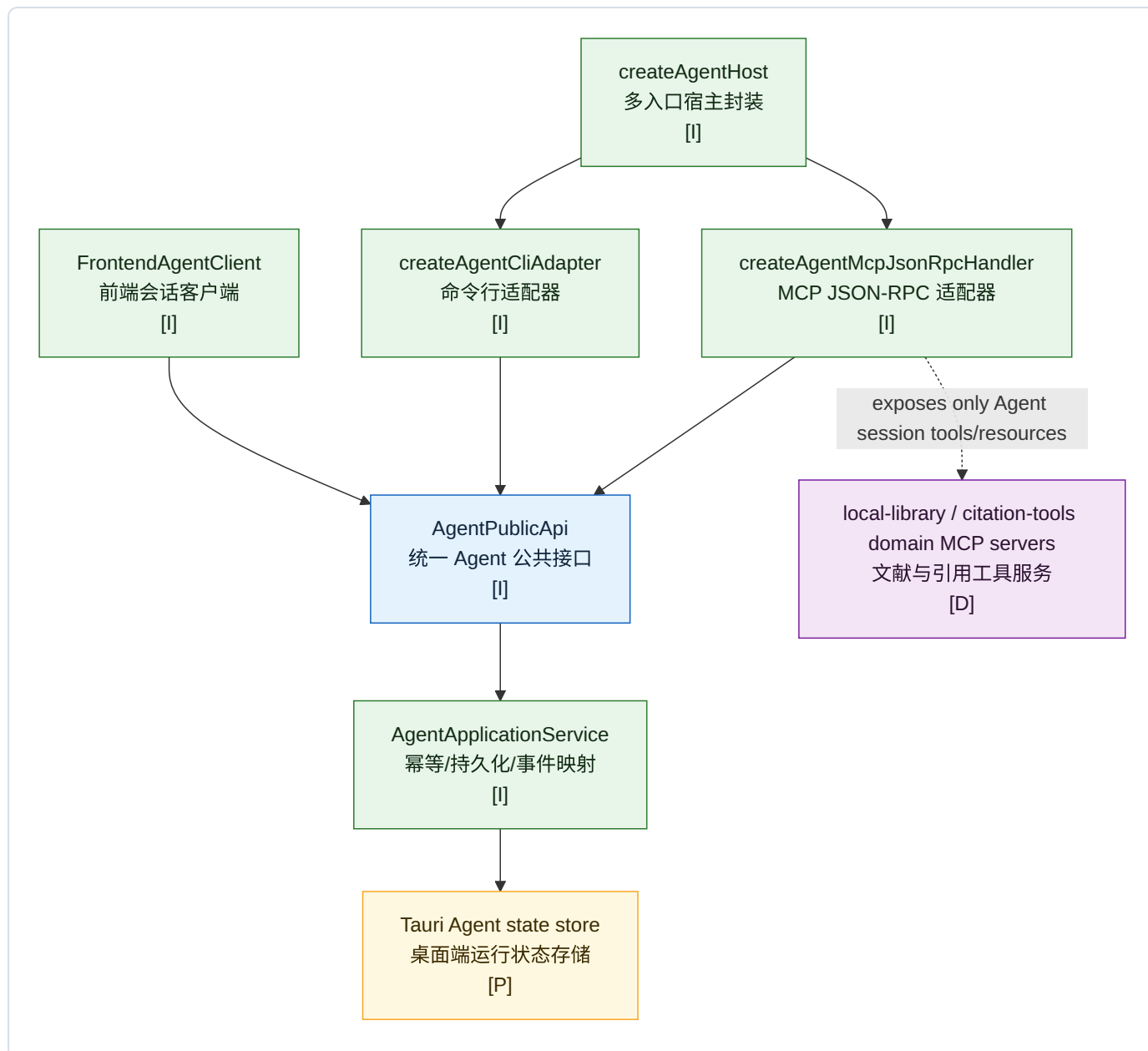


当前边界判断：

- 已有 imported chunk、citation、evidence ID、audit score 和 model execution trace。
- 多论文分析会构造 evidence matrix，并可并行生成子任务记录。
- 检索不是完整向量检索系统；`retrieval` 仍保留 demo fallback。
- 真实模型质量依赖 `dev-cloud` 和模型策略配置。

6. Public API / CLI / MCP 边界

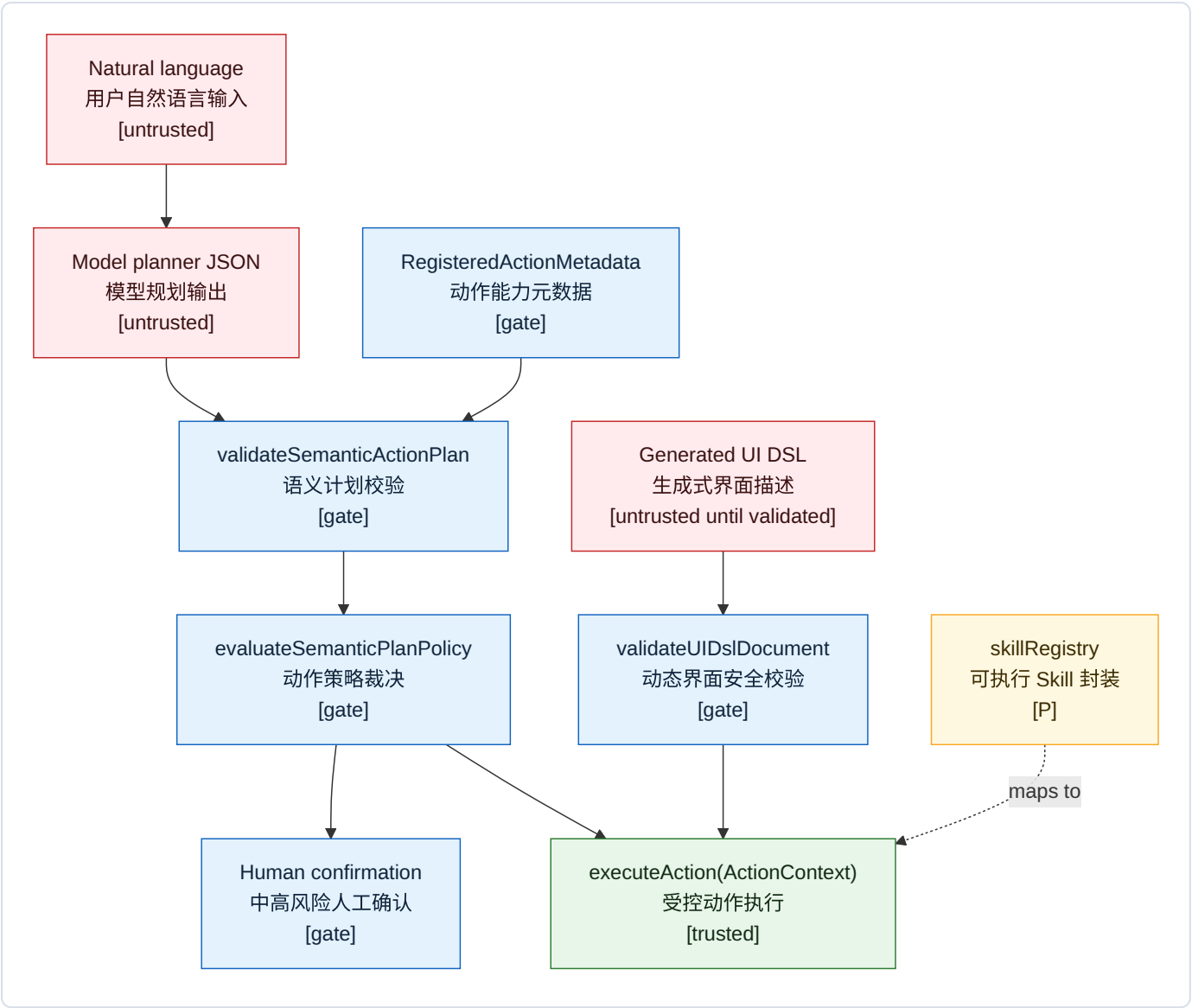
`agent-api` 是对外最重要的稳定边界。它把前端、CLI 和 MCP 统一到同一套 session/run/event 模型。



审计要点：

- MCP 当前暴露的是 Liteasy Agent host 工具：创建 session、提交 turn、确认、查询 run、取消 run。
- `agent-core` 配置里列出的 `local-library`、`citation-tools` 是未来领域 MCP server，不等同于当前已实现的 Agent MCP host。
- Public API 事件会把 runtime events 转成 `run.started`、`context.prepared`、`plan.preview`、`confirmation.required`、`assistant.message`、`ui.render` 等稳定事件。

7. Action / Skill / Policy 信任边界

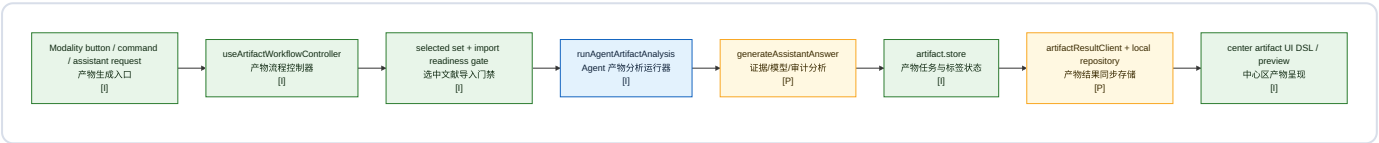


风险等级现状：

- 低风险：布局、面板、dock、主题、导入、推荐、收藏、组织资料区打开、artifact 生成等。
- 中风险： `settings.update` 针对 `profile.enabled` 会被动态提升为需要确认。
- 高风险：workspace 删除/覆盖/批量更新、cloud upload/sync 在 metadata 中标为高风险且需要确认。
- 高风险 action 当前没有真实执行 handler， `executeAction` 会拒绝并要求 approved high-risk handler；这是合理的保护默认值。

8. Artifact 生成链路

Artifact 是当前把 Agent 输出落到工作台主体验的关键路径。



当前特点：

- Artifact 生成通过 Agent Public API 发起 QA turn，而不是绕过 Agent 服务直接调模型。
- 生成过程会将 run 事件转为任务进度、流式 partial answer、SubAgent 工作记录和最终 artifact tab。
- 成功落库要求 Agent run 返回可持久化的 AnalysisRun/Evidence/Claim 元数据。

9. 模块成熟度矩阵

模块	成熟度	当前职责	审计备注
agent-api	I	Public API、事件、run、确认、取消、CLI/MCP adapter	已是稳定外壳，适合继续作为跨端契约
agent-core	P	prepareTurn、prompt context、memory search、budget guard	进入主链路，但配置和治理仍偏静态
agent-runtime	I	semantic plan、schema validation、policy、execution、rollback、UI DSL feedback	Command Mode 主链路清晰
actions/actionRegistry	I	action schema、risk、semantic frames、handler dispatch	真正的执行信任边界
skills/skillRegistry	P	三个 skill 到 action 的薄映射	与 agent-core skill catalog 有明显成熟度差异
assistant	P	对话 UI、Public API client 消费、确认、UI DSL 渲染	仍保留局部本地 runtime 兼容路径
retrieval	P	imported chunks、demo KB、citation payload	需要升级为生产检索/索引能力
paper-analysis	P	多论文 evidence matrix、claim、coverage	结构良好，但依赖检索质量
models	P	model policy、mock/dev-cloud/http proxy	策略门存在，生产稳定性依赖 dev-cloud
generative-ui	I	UI DSL generation、validation、dynamic action refs	安全边界明确，适合继续扩展组件卡
artifacts	I/P	task、tab、catalog、本地/服务结果同步	用户价值闭环已形成，持久化和服务端仍需硬化
plugins	D	plugin catalog	目前没有真实 plugin runtime
domain MCP servers	D	local-library、citation-tools 等	当前仅为 agent-core config 中的规划项
long-term memory	P/D	in-memory search and default memories	有 store 但缺少用户审查、持久化、注入安全扫描

10. 主要架构风险

R1. Skill Catalog 与可执行 Skill Registry 不一致

`agent-core` 默认配置列出多个 skill，并给出 active/planned/review 状态；`skillRegistry` 目前只可执行 `settings.adjust`、`artifact.generate`、`organization.open_shared_library` 三类。Command Mode 实际主要绕过 skill，直接规划到 action。

影响：团队容易误以为所有 active skill 都有独立执行器。

建议：明确 skill 的产品含义。若 skill 是“语义说明 + action frame”，就把命名改为 capability guide；若 skill 是可调用工具，就补齐统一 `executeSkill` contract。

R2. AssistantPane 仍承担较多 Agent 客户端编排

当前前端已经接入 `FrontendAgentClient` 和 Public API 事件，但 `AssistantPane` 仍有本地 runtime event formatter、local confirmation fallback、dynamic UI action direct execution 等兼容逻辑。

影响：后续新增 Agent 事件、确认或 UI DSL action 时，容易出现 Public API 路径和本地路径行为不一致。

建议：把 UI DSL dynamic action、confirmation、journal audit 消费继续收敛到 `agent-api` 或 dedicated controller，让 `AssistantPane` 更接近纯渲染/交互面。

R3. Agent Core 已在路径中，但治理面仍是静态样板

`agent-core.prepareTurn()` 已真正装配 prompt、memory、budget；但 `agent.md`、plugin、MCP server、memory catalog 多数来自 default config。

影响：架构上看起来已经有完整 Agent Core，但用户/管理员尚不能真正治理这些能力。

建议：优先做可持久化 `agent.md`、skill/capability 状态、memory 审查和禁用，而不是先扩更多模型调用能力。

R4. 检索链路还不足以支撑“准确、高性能、可追溯”的最终目标

当前 evidence/citation 结构已经不错，但 `retrieval` 仍有 demo fallback，真实 ingestion、chunk index、ranking、引用定位需要继续加强。

影响：Agent 输出可信度上限受限，尤其是多论文比较和深层树形展开。

建议：把 retrieval/ingestion 做成明确 owner 的生产链路：解析、切块、索引、召回、重排、citation 验证各自可测。

R5. 高风险 action 只有注册和保护，没有业务处理闭环

workspace 删除/覆盖、cloud upload/sync 等高风险动作已在 metadata 里建模并要求确认，但 `executeAction` 当前拒绝执行。

影响：这是安全默认值，但如果产品层展示了这些能力，用户会遇到“确认后仍无法执行”。

建议：继续保持默认拒绝，直到资源权限、撤销/回滚、审计日志和组织策略齐备后，再逐个接入 handler。

11. 下一步分工建议

按架构边界分配，而不是按页面分配。

工作流	推荐负责人	目标
Agent Core Governance	Agent 架构负责人	持久化 agent.md、memory 审查、capability/skill 启停、预算可视化
Runtime Contract Hardening	Runtime 负责人	收敛 planner、validator、policy、confirmation、rollback 的契约和测试
Assistant/API Convergence	前端架构负责人	把 AssistantPane 的本地 Agent 兼容逻辑迁移到 Public API/controller
Retrieval/Ingestion	检索负责人	真实 PDF 解析、chunk index、召回/重排、citation verifier
Artifact Pipeline	产物负责人	AnalysisRun/Evidence/Claim 持久化、artifact result service、失败恢复
MCP/Plugin Boundary	扩展负责人	区分 Agent host MCP 与 domain MCP server，设计插件权限与沙箱
Safety/Governance QA	QA/安全负责人	高风险 action 确认、拒绝、取消、审计日志、组织 namespace 隔离测试

12. 审计问题清单

下次架构评审建议直接检查这些问题：

1. 一个自然语言命令最终是否只能通过注册 action 改状态？
2. 每个 registered action 是否有 ownerFeature、schema、risk、confirmation、handler 和测试？
3. active skill 是否真的可执行，还是只是一段提示词/文档？
4. agent.md、memory、plugin、MCP 配置是否可被用户/组织治理，而不是写死在默认配置？
5. 模型 planner 失败时是否一定降级到 deterministic plan 或 clarification，而不是执行脏 JSON？
6. UI DSL 是否只能引用注册组件、注册数据源和注册 action？
7. QA / artifact 的事实结论是否能追溯到 evidence ID、paperId、page 和 snippet？
8. 高风险 action 是否具备确认、权限、审计、取消/回滚和失败恢复？
9. Public API、MCP、CLI、前端是否共享同一套 session/run/event 语义？
10. AssistantPane 是否还持有应该属于 controller 或 Agent service 的业务编排？

13. 推荐的近期验收口径

短期不要把目标定成“完整通用 Agent”。更合理的验收口径是：

LiteasyClaw P1 Agent = 学术工作台受控 Agent

必须做到：

- 会话/run/event/confirmation/cancel 统一；
- command 只能执行注册 action；
- QA/artifact 必须带证据和审计；
- agent-core 上下文进入每轮调用；
- skill/capability 状态可审计；
- 高风险动作默认拒绝或确认后执行受控 handler。

- 暂不要求：
- 任意 shell；
 - 任意本地文件读取；
 - 任意第三方 plugin 执行；
 - 完整外部 MCP 工具生态；
 - 无人监管的高风险写操作。

14. 与 BrainPilot Agent 手册对照

对照 `project-docs/reference/BrainPilot-Agent`构建学习手册.pdf 后，当前 LiteasyClaw 的方向是对的，但成熟度不能按“完整多 Agent 系统”宣传。更准确的定位是：

- 一个面向学术工作台的主 Agent Host
- + 确定性 runtime
 - + 多条模式化链路
 - + 局部并行分析子任务

它符合 BrainPilot 手册里“不要默认 Agent 越多越好”和“生产系统通常适合混合架构”的判断。当前阶段继续做单一主 Agent 更稳：上下文统一、权限边界集中、状态更容易审计，避免为了显得高级而拆出多个角色造成成本、延迟和冲突。

BrainPilot 原则	LiteasyClaw 当前状态	判断
单 Agent 适合目标集中、上下文统一的任务	LiteasyClaw 的主要目标是文献工作台内的问答、解释、命令和产物生成	匹配，当前不必强拆多 Agent
混合架构由代码控制关键阶段，Agent 在阶段内自主行动	<code>agent-runtime</code> 用 planner、validator、policy、executor 控制命令链路	匹配，这是当前最正确的架构方向
多 Agent 需要不同权限、上下文隔离、真实并行、独立审查或独立预算	当前只有 artifact 分析里的并行子任务，没有独立 mailbox、独立权限和 per-agent 状态	尚未达到 BrainPilot 式多 Agent
HTTP 应是命令入口 + 事件出口	<code>agent-api</code> 已有 session/run/event/confirmation/cancel 语义	方向匹配，但事件持久化和恢复还需加强
权限不能只靠 prompt，必须通过工具注册限制	Command Mode 最终只能执行 registered action，并经过 schema、policy、confirmation	匹配，是安全架构亮点
Prompt、Skill、Tool、Knowledge、Memory 要分层	LiteasyClaw 已有这些概念，但 <code>skillRegistry</code> 与 <code>agent-core catalog</code> 成熟度不一致	需要收敛命名和执行契约
多 Agent 需要 mailbox、背压、幂等、每 Agent 串行、跨 Agent 并行	当前没有完整 mailbox/delivery loop/backpressure/per-agent loop	暂不能称为完整多 Agent runtime
可靠性要有 durable job ledger、trace、预算、重试、评测基线	当前有 run/event、budget、audit 的雏形	需要作为 P1/P2 工程硬化重点

当前到底是几个 Agent？

从产品视角看，是一个 Liteasy Agent。

从工程视角看，是一个 `AgentPublicApi` / `AgentApplicationService` / `agent-core` 组成的主 Agent Host，下面分出几条受控链路：

1. Command Mode：自然语言到 action plan，再经过校验、策略、确认和执行，基本是串行链路。
2. QA / Explain：上下文装配、检索、模型回答、审计、UI DSL 输出，基本是串行链路。
3. Artifact Analysis：一个 Agent run 内部可以拆出多篇文章或多区段并行分析子任务，但这些子任务还不是独立 Agent。

所以当前不是“多个 Agent 互相协作”，而是“一条主 Agent 链路下有多个模式分支，部分分支内部可以并行调用模型”。

大模型 API 调用是并行还是串行？

当前应按链路区分：

- Command Mode：以串行为主。模型 planner 产出一个 plan 后，由 runtime 串行校验、裁决、执行。
- 普通 QA / Explain：以串行为主。检索和上下文准备后进入模型生成，再做审计和 UI 输出。
- 多论文 / Artifact 分析：可以在一个 run 内并行发起多个 paper/section 分析子任务，再汇总成最终回答或产物。

这更像“一个主链路，多模式分支，局部分支并行”，不是 BrainPilot 手册中“主协调 Agent + 多专家 Agent + mailbox 消息系统”的完整形态。

是否应该马上升级成多 Agent？

不建议现在就全面升级。只有当至少满足下面两项时，才值得拆出真正独立 Agent：

1. 不同角色必须拥有不同工具权限，例如检索者不能写 workspace，审计者不能改产物。
2. 某类任务需要隔离大上下文，例如全文检索、公式核查、引用审计各自上下文很重。
3. 工作能真正并行，并且并行结果有稳定结构化交接格式。
4. 需要独立质量审查责任，例如 artifact 发布前必须由 auditor 审查。
5. 某类任务需要独立模型、预算、超时和失败策略。

近期更合理的演进顺序是：

1. 保留一个 Liteasy 主 Agent，先把 Agent Host / Public API / runtime contract 做硬。
2. 把 SubAgent 工作记录 这类命名改成 并行分析子任务，避免团队误判为真实多 Agent。
3. 补 durable job ledger、事件持久化、幂等、trace、预算可视化和恢复测试。
4. 先把 retrieval/ingestion/evidence/citation 做成生产链路。
5. 第一批真正的第二 Agent 可以是 Auditor Agent，只读、低权限、专门审查事实、引用和 artifact 质量。